

Survey on high-quality security code review comments

Dear Participant,

We are researchers from Tianjin University, and we are currently conducting a research project on what constitutes a high-quality security code review comment (SCRC).

Background: Code review is an important software quality assurance practice in which reviewers inspect code changes and provide comments to identify potential problems and suggest improvements. In security code review, review comments are particularly important because they help developers identify, understand, and address potential security issues in code changes. To investigate what constitutes a high-quality security code review comment (SCRC), we conducted a **data triangulation** study from three complementary perspectives. From the **academic perspective**, we reviewed **45 papers specifically related to security code review**. From the **industry perspective**, we examined **2 established code review and secure code review guidelines from Google and Microsoft**. From the developer community perspective, we summarized the content of 40 relevant web resources retrieved through online searches. By synthesizing the evidence from these three perspectives, we identified four attributes that may characterize a high-quality security code review comment: **Identification, Reason, Impact, and Solution (IRIS)**.

Task: In this survey, we invite you to evaluate the importance of the four summarized attributes for the quality of security code review comments (SCRCs), and to provide your perspectives on the characteristics or criteria that should be considered when assessing each attribute.

The survey takes approximately 10 minutes to complete. Your identity will not be disclosed in any research outputs. All information provided in this survey will be used solely for research purposes.

* Indicates required question

Part I Participant Information

1. What is your current occupation? *

Mark only one oval.

- ☐ Industry professional
- ☐ Freelance developer
- ☐ Academic researcher
- ☐ Undergraduate or graduate student
- ☐ Other:

2. What is your country/region of residence? *

3. How many years of software development experience do you have? *

Mark only one oval.

- ☐ Less than 1 year
- ☐ 1-3 years
- ☐ 3-5 years
- ☐ More than 5 years

4. Which programming languages do you regularly use or encounter in your work or research? *

Check all that apply.

- ☐ C / C++
☐ Java
☐ Python
☐ Go
☐ C#
☐ PHP
☐ Rust
☐ Ruby
☐ Other: _____

5. Do you have practical experience with security code review? *

Mark only one oval.

- ☐ Yes
☐ No

6. What is your gender? *

Mark only one oval.

- ☐ Male
☐ Female
☐ Prefer not to say

Part II Characteristics of high-quality code review comments

For each attribute, we will provide real-world examples extracted from GitHub pull requests to illustrate the type of information represented by that attribute.

In each example:

- **<patch>** shows the relevant code changes from the pull request that were being reviewed. In the patch, lines beginning with "+" indicate newly added code, while lines beginning with "-" indicate removed code.
- **<comment>** shows the corresponding code review comment provided by a reviewer on the code change.

Please note that a real-world review comment may contain more than one type of information.

A. Identification refers to whether the comment identifies what the security issue is and where it occurs in the code change.

The following two real-world security code review examples illustrate the difference in the **Identification** information provided by review comments

- **Example 1**

This example is from the **Telegraf** project and concerns the error-handling code for an HTTP request using Basic Authentication.

<patch>:

```
1. + req.SetBasicAuth(n.Username, n.Password)
2. + resp, err = client.Do(req)
3. + if err != nil {
4. +     return fmt.Errorf("error making Basic Auth HTTP request to %s
   (USER:%s|PASSWORD:%s): %s\n", addr.String(), n.Username, n.Password, err)
```

<comment>: don't put the username & password in the error message, that will get logged and it's a security risk.

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file.

Patch & Comment: <https://patch1>

Code: <https://code1>

- **Example 2**

This example is from the **Ethersphere Swarm** project and concerns handling GetBlockHeaders requests received from peers.

<patch> :

```
1. + func (b *BzzEth) handleGetBlockHeaders(ctx context.Context, p *Peer, msg
   *GetBlockHeaders) {
2. +     p.logger.Debug("bzzeth.handleGetBlockHeaders", "id", msg.Rid, "hash",
   hex.EncodeToString(msg.Hashes[0]))
```

<comment> : Possible panic, even allows to a simple attack.

You can also click the URLs below to view the complete patch with review comment and the corresponding code file:

Patch & Comment: <https://patch2>

Code: <https://code2>

Comparison: Example 1 clearly identifies the problematic information and where it appears in the code. Example 2 warns about a possible panic and attack, but does not identify the

specific problematic expression (msg.Hashes[0]) or the condition that triggers it. This difference illustrates **Identification**.

7. **Question:** To what extent do you think **Identification** is important to the quality of a security code review comment (SCRC)? *

Mark only one oval.

- ☐ Very important
- ☐ Somewhat important
- ☐ Neutral
- ☐ Somewhat unimportant
- ☐ Very unimportant

8. What specific criteria would you use to evaluate the **Identification** of a SCRC? Please describe any characteristics or indicators that you consider important.

B. Reason refers to whether the comment explains why the identified issue occurs or why the code is problematic.

The following two real-world security code review examples illustrate the difference in the **Reason** information provided by review comments.

- **Example 1**

This example is from the **OpenTelemetry .NET** project and concerns incrementing a shared metric point index during metric processing.

<patch> :

1. + // check again after acquiring lock.
2. + if (!value2metrics.TryGetValue(tagValue, out aggregatorIndex)) {
3. + this.metricPointIndex++;

<comment> : This needs to be an Interlocked.Increment. The lock taken here is by key & value, multiple threads could be executing this at the same time and ++ is not atomic. ++ does a read and a write (2 operations). Read is atomic, write is atomic, but they are not atomic together. 99% sure about this.

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https://patch1>

Code: <https://code1>

- **Example 2**

This example is from the **Soletta** project and concerns data appended to an HTTP response buffer.

<patch> :

1. + r = sol_buffer_append_bytes(&response->content,
2. + (uint8_t *)mdata->value.blob->mem, mdata->value.blob->size);
3. + SOL_INT_CHECK(r, < 0, r);
- 4.
5. + if (!sol_http_param_add(&response->param, SOL_HTTP_REQUEST_PARAM_HEADER(
6. + HTTP_HEADER_CONTENT_TYPE, HTTP_HEADER_CONTENT_TYPE_JSON))) {
7. + SOL_WRN("Could not set the Content-Type:application/json");
8. + return -ENOMEM;

<comment> : response.content will leak. You need to call sol_buffer_fini

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https://patch2>

Code: <https://code2>

Comparison: Example 1 explains why the issue occurs by describing the concurrent execution and non-atomic ++ operation. Example 2 identifies a memory leak and suggests a fix, but does not explain why the leak occurs.

9. **Question:** To what extent do you think **Reason** is important to the quality of a security code review comment (SCRC)? *

Mark only one oval.

- ☐ Very important
- ☐ Somewhat important
- ☐ Neutral
- ☐ Somewhat unimportant
- ☐ Very unimportant

10. What specific criteria would you use to evaluate the **Reason** of a SCRC? Please describe any characteristics or indicators that you consider important.

C. Impact refers to whether the comment describes what negative consequences or security risks may result from the identified issue.

The following two real-world security code review examples illustrate the difference in the **Impact** information provided by review comments.

- **Example 1**

This example is from the **Cosmos SDK** project and concerns decoding transactions while processing a block proposal.

<patch> :

```
1. + for _, txBytes := range req.Txs {  
2. +     tx, err := app.txDecoder(txBytes)  
3. +     if err != nil {  
4. +         panic(err)
```

<comment> : This doesn't seem right to me. If a byzantine proposer sends you malformed TXs that you can't decode, you shouldn't crash. That's DoS. Shouldn't you return "Reject" here?

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https://patch1>

Code: <https://code1>

- **Example 2**

This example is from the **OpenSSL** project and concerns resizing a buffer while building a list of supported signature algorithms.

<patch> :

```
1. + if (strlen(sa) + strlen(retval) + 1 >= maxretlen) {  
2. +     maxretlen += SIGLEN_BUF_INCREMENT;  
3. +     retval = OPENSSL_realloc(retval, maxretlen);  
4. + }
```

<comment> : Please check the return value for NULL, also release the original value in such case and return error (i.e. NULL).

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https://patch2>

Code: <https://code2>

Comparison: Example 1 explicitly describes the consequence of the issue: a malicious proposer could crash the node and cause DoS. Example 2 identifies an allocation failure and how to handle it, but does not explain what may happen if it is not handled.

11. **Question:** To what extent do you think **Impact** is important to the quality of a security code review comment (SCRC)? *

Mark only one oval.

- ☐ Very important
- ☐ Somewhat important
- ☐ Neutral
- ☐ Somewhat unimportant
- ☐ Very unimportant

12. What specific criteria would you use to evaluate the **Impact** of a SCRC? Please describe any characteristics or indicators that you consider important.

D. Solution refers to whether the comment provides useful and actionable guidance for addressing the identified issue.

The following two real-world security code review examples illustrate the difference in the **Solution** information provided by review comments.

- **Example 1**

This example is from the **Telegraf** project and concerns obtaining a database name using a Go type assertion.

<patch> :

```
1. + if columnMap["database"] != nil {
2. +     // extract the database name from the column map
3. +     _, err := dbname.WriteString((*columnMap["database"]).(string))
4. +     if err != nil {
5. +         return nil, nil, err
```

<comment> : Honestly speaking I'm a bit scared to assume a sting here without checking it. This line will make Telegraf panic if the column does not contain a string! So how about:

```
1. name, ok := *columnMap["database"].(string).
2. if !ok {
3.     return nil, nil, fmt.Errorf("database not a string but %T", *columnMap["database"])
```

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https://patch1>

Code: <https://code1>

- **Example 2**

This example is from the **Apache Pinot** project and concerns logging values obtained from environment variables.

<patch> :

```
1. + if (envVar != null)
2. + {
    configuration.setProperty(dynamicEnvConfigVarName, envVar);
3. +  LOGGER.info(
    "The environment variable {} is set to the property {} through dynamic configs",
    envVar,
    dynamicEnvConfigVarName);
    }
```

<comment> : There's a risk that config in env variables might be used to store secrets. This would leak them in the logs.

You can also click the URLs below to view the complete patch and review comment, as well as the corresponding code file:

Patch & Comment: <https:patch2>

Code: <https://code2>

Comparison: Example 1 provides concrete code showing how to address the issue. Example 2 clearly identifies the security risk, but does not provide guidance on how it should be fixed.

13. **Question:** To what extent do you think **Solution** is important to the quality of a security code review comment (SCRC)? *

Mark only one oval.

- ☐ Very important
- ☐ Somewhat important
- ☐ Neutral
- ☐ Somewhat unimportant
- ☐ Very unimportant

14. What specific criteria would you use to evaluate the **Solution** of a SCRC? Please describe any characteristics or indicators that you consider important.

Part III Open-ended Question

15. Are there any important aspects of high-quality SCRCs that are not covered by Identification, Reason, Impact, and Solution? If so, please describe them.

16. If you have any additional comments, suggestions, or thoughts, please feel free to share them below.

This content is neither created nor endorsed by Google.

Google Forms

